Zcim Project

Digital Research CP/M®

Basic Disk Operating System (BDOS)

Specifications

draft 2025-09-04

Copyright

Copyright © 1976, 1977, 1978, 1979, 1982, and 1983 by Digital Research. All rights reserved. No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form or by any means, electronic, mechanical, optical, chemical, manual or otherwise, without the prior written permission of ~~Digital Research, Post Office Box 579, Pacific Grove, California 93950~~.

~~http://www.lineo.com~~

DRDOS, Inc [Bryan Sparks]

Copyright © 2025 by James Burlingame.

License

The original Digital Research Inc assets license is available at http://cpm.z80.de/license.html.

This *Zcim Project edition* is licensed under the Creative Commons Attribution 4.0 International license. **CC BY 4.0**.

Disclaimer

Digital Research makes no representations or warranties with respect to the contents hereof and specifically disclaims any implied warranties of merchantability or fitness for any particular purpose. Further, Digital Research reserves the right to revise this publication and to make changes from time to time in the content hereof without obligation of Digital Research to notify any person of such revision or changes.

Trademarks

CP/M and CP/NET are registered trademarks of Digital Research. ASM, DESPOOL, DDT, LINK-80, MAC, MP/M, PL/I-80, and SID are trademarks of Digital Research. Intel is a registered trademark of Intel Corporation. TI Silent 700 is a trademark of Texas Instruments Incorporated. Zilog and Z80 are registered trademarks of Zilog, Inc.

Printing

Typst Edition: draft 2025-09-04

Contents

List of Tables

1. Introduction

This document is a set of reverse engineered specifications of the Basic Disk Operating System (BDOS) component of the CP/M Operating System. These specifications are related to the 8-bit version of CP/M, that was in later years called CP/M-80. Other versions of CP/M on 16-bit and 32-bit processors are not included.

Similarly, MP/M is a related, but distinct product and is not included in these specifications.

The goal of these specifications is to provide enough information to reproduce, or at least imitate, the behavior of CP/M as part of the Zcim Project.

1.1. Known Versions

Since CP/M is a legacy product, the known versions are largely fixed. Copies of these versions are available. Other versions are known to have existed but are not available (e.g. `2.1`).

Id	Description
`1975`	An early version from 1975 - supported 2 drives, combined FDOS
`1.3`	CP/M 1.3
`1.4`	CP/M 1.4 - supports 4 drives
`2.0`	CP/M 2.0 - first 2.x release, BDOS/BIOS split, 16 drive support, random I/O
`2.2`	CP/M 2.2 - fixes bugs in 2.0, adds function 40
`3.0`	CP/M 3.0 aka CP/M Plus - nonbanked - without banked-memory
`3.0b`	CP/M 3.0 aka CP/M Plus - banked - with banked-memory

Table 1: Known CP/M Versions

1.2. External Interfaces

The BDOS has three interfaces:

- Basic Input/Output System - BIOS
- Console Command Processor - CCP
- System Call Interface (`CALL 0005H`)

1.3. Memory Layout

The memory layout used by CP/M evolved in each major version.

1.3.1. Original Version 1 Memory Layout

Memory organization of the original CP/M system is shown in Table 2

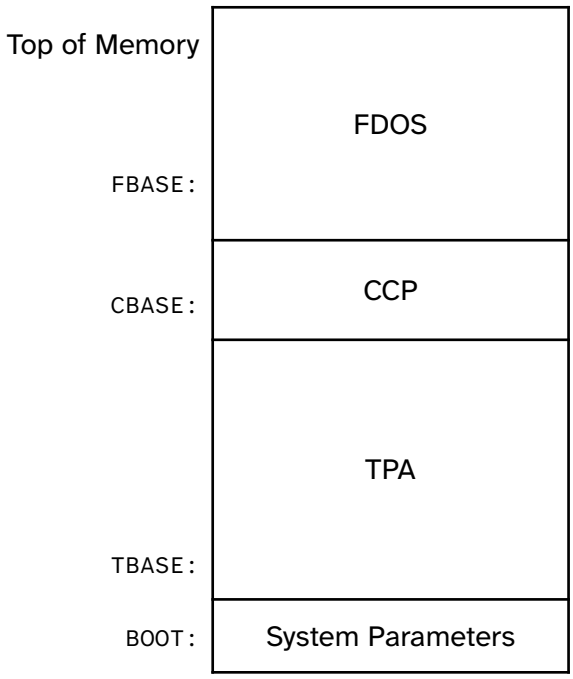


Table 2.: CP/M 1 Memory Organization

1.3.2. CP/M Version 2 Memory Layout

The major change in version 2 was the introduction of the BDOS/BIOS interface. While the existing FDOS view had minimal changes from version 1, there was an additional division of the FDOS into a Basic Disk Operating System (BDOS), and a separate Basic Input/Output System (BIOS). The BDOS was the same for all CP/M systems, while the BIOS contained system specific portions, which were customized for most systems.

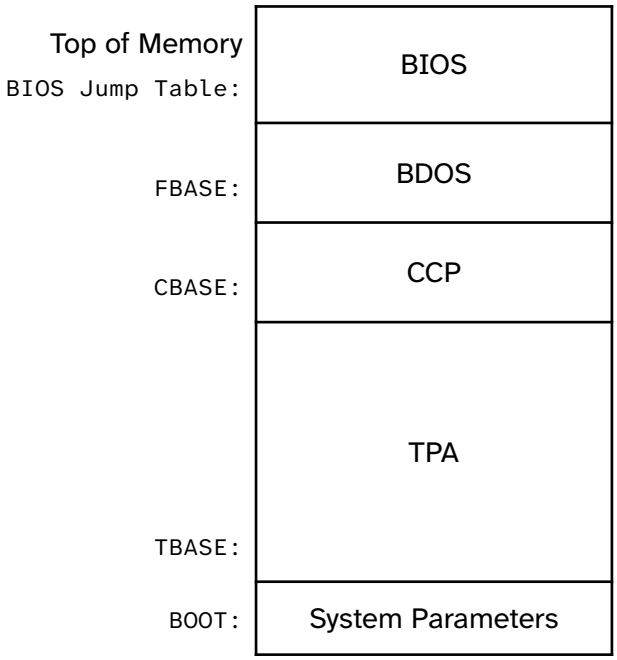


Table 3.: CP/M 2 Memory Organization

1.3.3. CP/M 3 Memory Layout

There were two options in the version 3 memory layout, since it supported **bank switching** in order to break through the dreaded 64KiB memory barrier.

> Of course, at that time marketing had not perverted the concept of a **K**, so it was always **64K**, since **KiB** had not been developed to keep pedantic fools like me happy.

1.3.3.1. Non-Banked System

In a non-banked system, you still had to live with the 64K limit while still trying to add more features.

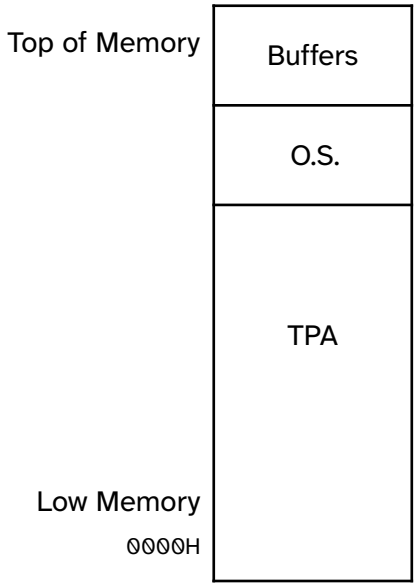
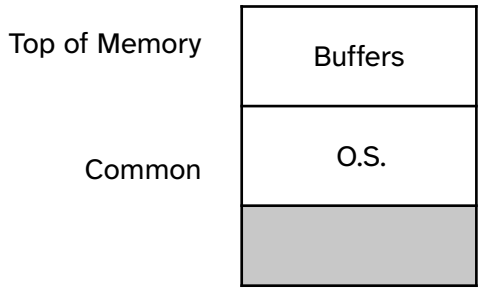


Table 4.: CP/M 3 Non-Banked Memory Organization

1.3.3.2. Banked-memory System



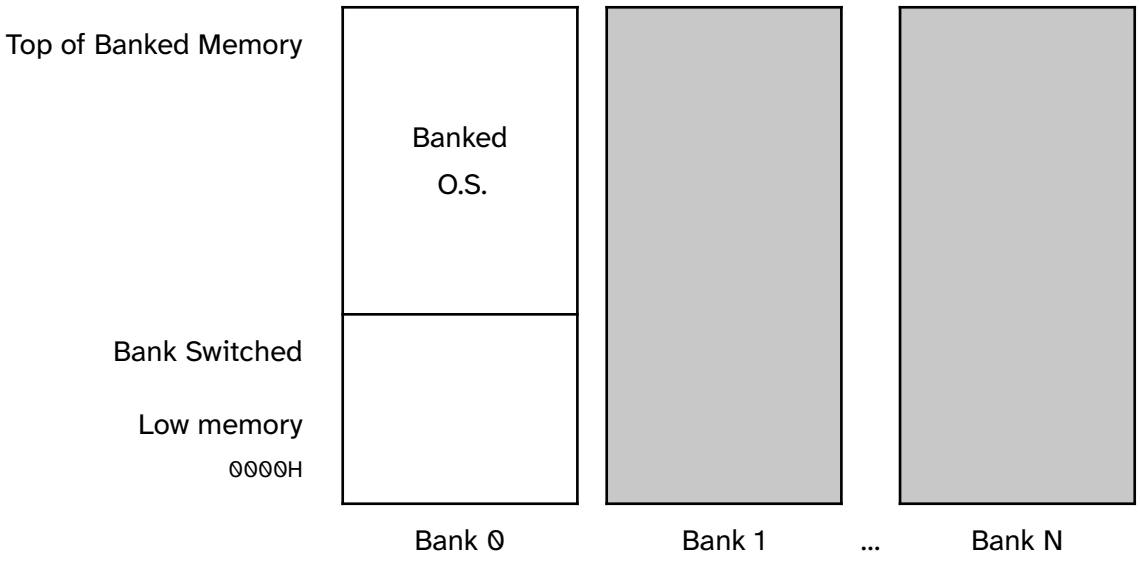


Table 5.: CP/M 3 Banked Memory Organization

Field	DR	F1 … F8	T1	T2	T3	EX	S1	S2	RC	D0 … D15	CR	R0	R1	R2
Decimal	00	01 … 08	09	10	11	12	13	14	15	16 … 31	32	33	34	35
Hex	00	01 … 08	09	0A	0B	0C	0D	0E	0F	10 … 1F	20	21	22	23

Table 6.: File Control Block Format

Field	Definition
DR	drive code (0-16). 0=default, 1=Drive A:, 2=B:, … 16=P:
F1 … **F8**	contain the filename in ASCII upper-case, with high bits = `f1'` … `f8'` normally zero
T1	contains the first character of the filetype in ASCII upper-case, with high bit = `t1'`. Set for Read/Only file.
T2	contains the second character of the filetype in ASCII upper-case with high bit = `t2'`. Set for SYS file, no DIR list.
T3	contains the third character of the filetype in ASCII upper-case with high bit = `t3'` normally zero.
EX	contains the current extent number, normally set to 00 by the user, but in range 0-31 during file I/O
S1	reserved for internal system use
S2	reserved for internal system use. Set to zero on calls to **Open**, **Make** and **Search for First**
RC	record count for extent **EX**; takes on values from 0-127
D0 … **D15**	filled in by CP/M; reserved for system use
CR	current record to read or write in a sequential file operation; normally set to zero by user
R0	Optional Random Record Number low byte
R1	Optional Random Record Number high byte
R2	Optional Random Record Number overflow

Table 7.: File Control Block Fields

Each file being accessed through CP/M must have a corresponding FCB, which provides the name and allocation information for all subsequent file operations. When accessing files, it is the programmer's responsibility to fill the lower 16 bytes of the FCB and initialize the **CR** field. Normally, bytes 1 through 11 are set to the ASCII character values for the filename and filetype, while all other fields are zero.

FCBs are stored in a directory area of the disk, and are brought into central memory before the programmer proceeds with file operations (see the **Open** and **Make** functions). The memory copy of the FCB is updated as file operations take place and later recorded permanently on disk at the termination of the file operation, (see the **Close** command).

The CCP constructs the first 16 bytes of two optional FCBs for a transient by scanning the remainder of the line following the transient name, denoted by *file1* and *file2* in the prototype command line described above, with unspecified fields set to ASCII blanks. The first FCB is constructed at location BOOT+005CH and can be used as is for subsequent file operations. The second FCB occupies the **D0** ... **Dn** portion of the first FCB and must be moved to another area of memory before use. If, for example, the following command line is typed:

```
PROGNAME B:X.ZOT Y.ZAP
```

the file PROGNAME.COM is loaded into the TPA, and the default FCB at BOOT+005CH is initialized to drive code 2, filename X, and filetype ZOT. The second drive code takes the default value 0, which is placed at BOOT+006CH, with the filename Y placed into location BOOT+006DH and filetype ZAP located 8 bytes later at BOOT+0075H. All remaining fields through **CR** are set to zero. Note again that it is the programmer's responsibility to move this second filename and filetype to another area, usually a separate file control block, before opening the file that begins at BOOT+005CH, because the open operation overwrites the second name and type.

If no filenames are specified in the original command, the fields beginning at BOOT+005DH and BOOT+006DH contain blanks. In all cases, the CCP translates lower-case alphabetic characters to upper-case to be consistent with the CP/M file naming conventions

As an added convenience, the default buffer area at location BOOT+0080H is initialized to the command line tail typed by the operator following the program name. The first position contains the number of characters, with the characters themselves following the character count. Given the above command line, the area beginning at BOOT+0080H is initialized as follows:

Offset	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90 ... FF
Content	0EH	*sp*	'B'	':'	'X'	'.'	'Z'	'O'	'T'	*sp*	'Y'	'.'	'Z'	'A'	'P'	00H	???

Table 8.: Command-line Layout

where the characters are translated to upper-case ASCII with uninitialized memory following the last valid character. Again, it is the responsibility of the programmer to extract the information

from this buffer before any file operations are performed, unless the default DMA address is explicitly changed.

2. CP/M System Calls

The CP/M System Call is initiated by executing a CALL 0005H instruction. The calling convention has at least one parameter, which is passed in the C register. It is the BDOS Function number. Additional parameters are either passed in the E register, the DE register pair. Many BDOS functions also depend upon, and alter, shared state that CP/M maintains.

Individual functions are described in detail in the pages that follow. For each function, to start, there is a table. In the table, the entry parameters are described, along with any returned values. If global state is used, that is described. Also, if there differences between versions, that is summarized. After the table, there is more detail about the operation of the specific function.

Unless otherwise indicated, the value returned to the user code is in two sets of registers. A single byte value is returned in register A, which is also returned in register L. A two-byte value is returned in the HL register-pair as well as register B (msb), and register A (lsb).

No input registers are preserved.

Flags are not defined after a BDOS System Call.

2.1. System Reset

Function 0: System Reset

Entry Parameters:	
Register C:	0 = 00H
Returned Value:	**Does not return**
Versions supported:	All versions

The **System Reset** function returns control to the CP/M operating system at the CCP level. The CCP re-initializes the disk subsystem by selecting and logging-in disk drive A. This function has exactly the same effect as a jump to location 0000H (BOOT).

2.2. Console Input

Function 1: Console Input

Entry Parameters:	
Register C:	1 = 01H
Returned Value:	
Register A:	ASCII Character
Versions supported:	All versions

The **Console Input** function reads the next console character to register A. Graphic characters, along with carriage return, line-feed, and back space (CTRL-H) are echoed to the console. Tab characters, CTRL-I, move the cursor to the next tab stop. A check is made for start/stop scroll,

CTRL-S, and start/stop printer echo, CTRL-P. The FDOS does not return to the calling program until a character has been typed, thus suspending execution if a character is not ready.

2.3. Console Output

Function 2: Console Output

Entry Parameters:	
Register C:	2 = 02H
Register E:	ASCII character to output
Returned Value:	**none**
Typical PL/M Call:	`CALL MON1(2, 'A')`
Global State:	
`console-column`	updated
`MultipleCommandBufferPage`	really long name
Versions supported:	All versions
Version Specific:	
CP/M 1.3	really cool stuff
CP/M 2.0	IOBYTE functionality
CP/M 3 (CP/M Plus)	Now with console redirection instead of IOBYTE
CP/M 3 (CP/M Plus) non-banked memory system	Does nothing
CP/M 3 (CP/M Plus) banked memory system	Does something else

The **Console Output** function sends the ASCII character from register E to the console device. As in Function 1, tabs are expanded and checks are made for start/stop scroll and printer echo.

2.4. Reader Input

Function 3: Reader Input

Entry Parameters:	
Register C:	3 = 03H
Returned Value:	
Register A:	ASCII Character
Versions supported:	All versions

The **Reader Input** function reads the next character from the logical reader into register A. Control does not return until the character has been read.

2.5. Punch Output

Function 4: Punch Output

Entry Parameters:	

Register C:	4 = 04H
Register E:	ASCII Character
Returned Value:	**none**
Versions supported:	All versions

The **Punch Output** function sends the character from register E to the logical punch device.

2.6. List Output

Function 5: List Output

Entry Parameters:	
Register C:	5 = 05H
Register E:	ASCII Character
Returned Value:	**none**
Versions supported:	All versions

The **List Output** function sends the ASCII character in register E to the logical listing device.

2.7. Direct Console I/O

Function 6: Direct Console I/O

Entry Parameters:	
Register C:	6 = 06H
Register E:	0FFH (input) or char (output)
Returned Value:	
Register A:	character or status
Versions supported:	CP/M 2 and later

Direct Console I/O is supported under CP/M for those specialized applications where basic console input and output are required. Use of this function should, in general, be avoided since it bypasses all of the CP/M normal control character functions (for example, CTRL-S and CTRL-P). Programs that perform direct I/O through the BIOS under previous releases of CP/M, however, should be changed to use direct I/O under BDOS so that they can be fully supported under future releases of MP/M and CP/M.

Upon entry to Function 6, register E either contains hexadecimal 0FFH, denoting a console input request, or an ASCII character. If the input value is 0FFH, Function 6 returns A = 00 if no character is ready, otherwise A contains the next console input character.

If the input value in E is not 0FFH, Function 6 assumes that E contains a valid ASCII character that is sent to the console.

2.8. Memory Size

Function 6: Memory Size

Entry Parameters:	
Register C:	6 = 06H
Returned Value:	
Registers HL:	Base Address of the CCP
Versions supported:	CP/M 1 only

The **Memory Size** function returns the base address of the Console Command Processor (CCP) in HL.

2.9. Auxiliary Input Status

Function 7: Auxiliary Input Status

Entry Parameters:	
Register C:	7 = 07H
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not
Versions supported:	CP/M 3 only

2.10. Get IOBYTE

Function 7: Get IOBYTE

Entry Parameters:	
Register C:	7 = 07H
Returned Value:	
Register A:	IOBYTE value
Versions supported:	CP/M 1 and CP/M 2

The **Get IOBYTE** function returns the current value of IOBYTE in register A.

2.11. Auxiliary Output Status

Function 8: Auxiliary Output Status

Entry Parameters:	
Register C:	8 = 08H
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not
Versions supported:	CP/M 3 only

2.12. Set IOBYTE

Function 8: Set IOBYTE

Entry Parameters:	
Register C:	8 = 08H
Register E:	IOBYTE value
Returned Value:	**none**
Versions supported:	CP/M 1 and CP/M 2

The **Set IOBYTE** function changes the IOBYTE value to that given in register E.

2.13. Print String

Function 9: Print String

Entry Parameters:	
Register C:	9 = 09H
Registers DE:	String Address
Returned Value:	**none**
Versions supported:	All versions

The **Print String** function sends the character string stored in memory at the location given by DE to the console device, until a '$' (24H) is encountered in the string. Tabs are expanded as in Function 2, and checks are made for start/stop scroll and printer echo.

2.14. Read Console Buffer

Function 10: Read Console Buffer

Entry Parameters:	
Register C:	10 = 0AH
Registers DE:	Buffer Address
Returned Value:	Characters input are in the Buffer
Versions supported:	All versions

The **Read Buffer** functions reads a line of edited console input into a buffer addressed by registers DE. Console input is terminated when either input buffer overflows or a carriage return or line-feed is typed. The Read Buffer takes the form:

DE:	+0	+1	+2	+3	+4	+5	+6	+7	+8	...	+n
	mx	**nc**	*c1*	*c2*	*c3*	*c4*	*c5*	*c6*	*c7*	...	??

where **m** is the maximum number of characters that the buffer will hold, 1 to 255, and **nc** is the number of characters read (set by FDOS upon return) followed by the characters read from the console. If $\mathbf{nc} < \mathbf{mx}$, then uninitialized positions follow the last character, denoted by ?? in the above figure. A number of control functions, summarized in Table 9, are recognized during line editing.

Table 9.: Edit Control Characters

Character	Meaning
CTRL-C	reboots when at beginning of line
CTRL-E	causes physical end of line
CTRL-H	backspaces one character position
CTRL-J	(line-feed) terminates input line
CTRL-M	(carriage return) terminates input line
CTRL-R	retypes the current line after new line
CTRL-U	removes current line
CTRL-X	Same as CTRL-U.
rubout/del	Deletes and echoes the last character typed at the console.

The user should also note that certain functions that return the carriage to the leftmost position (for example, CTRL-X) do so only to the column position where the prompt ended. In earlier releases, the carriage returned to the extreme left margin. This convention makes operator data input and line correction more legible.

2.15. Get Console Status

Function 11: Get Console Status

Entry Parameters:	
Register C:	11 = 0BH
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not
Versions supported:	All versions

The **Console Status** function checks to see if a character has been typed at the console. If a character is ready, the value 0FFH is returned in register A. Otherwise a 00H value is returned.

2.16. Lift Head

Function 12: Lift Head

Entry Parameters:	
Register C:	12 = 0CH

Returned Value:	
Register A:	00H
Registers HL:	FCBDSK variable address or 0
Versions supported:	CP/M 1 only

2.17. Version Number

Function 12: Version Number

Entry Parameters:	
Register C:	12 = 0CH
Returned Value:	
Registers HL:	Version Number
Versions supported:	CP/M 2 and later

Function 12 provides information that allows version independent programming. A two-byte value is returned, with H = 00H designating the CP/M release (H = 01 for MP/M) and L = 00 for all releases previous to 2.0. CP/M 2.0 returns a hexadecimal 20 in register L, with subsequent version 2 releases in the hexadecimal range 21, 22, through 2F. Using Function 12, for example, the user can write application programs that provide both sequential and random access functions.

2.18. Reset Disk System

Function 13: Reset Disk System

Entry Parameters:	
Register C:	13 = 0DH
Returned Value:	
Register A:	0FFH if $ file present, 0 if not
Versions supported:	All versions

The **Reset Disk** function is used to programmatically restore the file system to a reset state where all disks are set to Read-Write. See functions 28 and 29, only disk drive A is selected, and the default DMA address is reset to BOOT+0080H. This function can be used, for example, by an application program that requires a disk change without a system reboot.

This function is normally called by the CCP after the system reset and before any other activity. If the CCP is not executed, this function must be called by what replaced it.

In 1.x and 2.x version the return value depends upon the presence of a file with a name starting with a dollar-sign ($) on drive A. If such a file is present, a 0FFH value is returned, otherwise a zero is returned.

2.19. Select Disk

Function 14: Select Disk

Entry Parameters:	
Register C:	14 = 0EH
Register E:	Drive number: 0 for A, 1 for B, ... 15 for P
Returned Value:	
Register A:	0 if successful, 0FFH on error
Versions supported:	All versions

The **Select Disk** function designates the disk drive named in register E as the default disk for subsequent file operations, with E = 0 for drive A, 1 for drive B, and so on through 15, corresponding to drive P in a full 16 drive system. The drive is placed in an on-line status, which activates its directory until the next cold start, warm start, or disk system reset operation. If the disk medium is changed while it is on-line, the drive automatically goes to a Read-Only status in a standard CP/M environment, see Function 28. FCBs that specify drive code zero (**DR** = 00H) automatically reference the currently selected default drive. Drive code values between 1 and 16 ignore the selected default drive and directly reference drives A through P.

2.20. Open File

Function 15: Open File

Entry Parameters:	
Register C:	15 = 0FH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Global State:	
dirbuf	contents updated
Versions supported:	All versions

The **Open** File operation is used to activate a file that currently exists in the disk directory for the currently active user number. The FDOS scans the referenced disk directory for a match in positions 1 through 14 of the FCB referenced by DE (byte **S1** is automatically zeroed) where an ASCII question mark '?' (3FH) matches any directory character in any of these positions. Normally, no question marks are included, and bytes **EX** and **S2** of the FCB are zero.

If a directory element is matched, the relevant directory information is copied into bytes **D0** through **Dn** of FCB, thus allowing access to the files through subsequent read and write operations. The user should note that an existing file must not be accessed until a successful open operation is completed. Upon return, the open function returns a directory code with the

value 0 through 3 if the open was successful or 0FFH (255 decimal) if the file cannot be found. If question marks occur in the FCB, the first matching FCB is activated. Note that the current record, (**CR**) must be zeroed by the program if the file is to be accessed sequentially from the first record.

2.21. Close File

Function 16: Close File

Entry Parameters:	
Register C:	16 = 10H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

The **Close** File function performs the inverse of the **Open** File function. Given that the FCB addressed by DE has been previously activated through an **Open** or **Make** function, the **Close** function permanently records the new FCB in the reference disk directory see functions 15 and 22. The FCB matching process for the close is identical to the open function. The directory code returned for a successful close operation is 0, 1, 2, or 3, while a 0FFH (255 decimal) is returned if the filename cannot be found in the directory. A file need not be closed if only read operations have taken place. If write operations have occurred, the close operation is necessary to record the new directory information permanently.

2.22. Search for First

Function 17: Search for First

Entry Parameters:	
Register C:	17 = 11H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

Search for First scans the directory for a match with the file given by the FCB addressed by DE. The value 255 (hexadecimal 0FFH) is returned if the file is not found; otherwise, 0, 1, 2, or 3 is returned indicating the file is present. When the file is found, the current DMA address is filled with the record containing the directory entry, and the relative starting position is $A * 32$ (that is, rotate the A register left 5 bits, or ADD A five times). Although not normally required for application programs, the directory information can be extracted from the buffer at this position.

An ASCII question mark '?' (63 decimal, 3F hexadecimal) in any position from **F1** through **EX** matches the corresponding field of any directory entry on the default or auto-selected disk drive. If the **DR** field contains an ASCII question mark, the auto disk select function is disabled and the default disk is searched, with the search function returning any matched entry, allocated or free, belonging to any user number. This latter function is not normally used by application programs, but it allows complete flexibility to scan all current directory values. If the **DR** field is not a question mark, the **S2** byte is automatically zeroed.

2.23. Search for Next

Function 18: Search for Next

Entry Parameters:	
Register C:	18 = 12H
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

The **Search for Next** function is similar to the **Search for First** function, except that the directory scan continues from the last matched entry. Similar to Function 17, Function 18 returns the decimal value 255 in A when no more directory items match.

2.24. Delete File

Function 19: Delete File

Entry Parameters:	
Register C:	19 = 13H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

The **Delete** File function removes files that match the FCB addressed by DE. The filename and type may contain ambiguous references (that is, question marks in various positions), but the drive select code (**DR**) cannot be ambiguous, as in the **Search for First** and **Search for Next** functions.

Function 19 returns a decimal 255 if the referenced file or files cannot be found; otherwise, a value in the range 0 to 3 returned.

2.25. Read Sequential

Function 20: Read Sequential

Entry Parameters:	

Register C:	20 = 14H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Read Sequential** function reads the next 128-byte record from the file into memory at the current DMA address. The record is read from position **CR** of the extent, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next read operation. The value 00H is returned in the A register if the read operation was successful, while a nonzero value is returned if no data exist at the next record position (for example, end-of-file occurs).

2.26. Write Sequential

Function 21: Write Sequential

Entry Parameters:	
Register C:	21 = 15H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Write Sequential** function writes the 128-byte data record at the current DMA address to the file named by the FCB. The record is placed at position **CR** of the file, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next write operation. Write operations can take place into an existing file, in which case, newly written records overlay those that already exist in the file. Register A = 00H upon return from a successful write operation, while a nonzero value indicates an unsuccessful write caused by a full disk.

2.27. Make File

Function 22: Make File

Entry Parameters:	
Register C:	22 = 16H
Registers DE:	FCB Address

Returned Value:	
Register A:	Directory Code
Versions supported:	All versions

The **Make** File operation is similar to the **Open** File operation except that the FCB must name a file that does not exist in the currently referenced disk directory (that is, the one named explicitly by a nonzero **DR** code or the default disk if **DR** is zero). The FDOS creates the file and initializes both the directory and main memory value to an empty file. The programmer must ensure that no duplicate filenames occur, and a preceding delete operation is sufficient if there is any possibility of duplication. Upon return, register A = 0, 1, 2, or 3 if the operation was successful and 0FFH (255 decimal) if no more directory space is available. The Make function has the side effect of activating the FCB and thus a subsequent open is not necessary.

2.28. Rename File

Function 23: Rename File

Entry Parameters:	
Register C:	23 = 17H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	All versions
Version Specific:	
CP/M 3 (CP/M Plus)	H with be 0 if the file was not found, a non-zero value indicates a hardware error code.

The **Rename** function uses the FCB addressed by DE to change all occurrences of the file named in the first 16 bytes to the file named in the second 16 bytes. The drive code **DR** at position 0 is used to select the drive, while the drive code for the new filename at position 16 of the FCB is assumed to be zero. Upon return, register A is set to a value between 0 and 3 if the rename was successful and 0FFH (255 decimal) if the first filename could not be found in the directory scan.

2.29. Return Login Vector

Function 24: Return Login Vector

Entry Parameters:	
Register C:	24 = 18H
Returned Value:	
Registers HL:	Log-in Vector
Versions supported:	All versions

The log-in vector value returned by CP/M is a 16-bit value in HL, where the least significant bit of L corresponds to the first drive A and the high-order bit of H corresponds to the sixteenth drive, labeled P. A 0 bit indicates that the drive is not on-line, while a 1 bit marks a drive that is actively on-line as a result of an explicit disk drive selection or an implicit drive select caused by a file operation that specified a nonzero **DR** field. The user should note that compatibility is maintained with earlier releases, because registers A and L contain the same values upon return.

2.30. Return Current Disk

Function 25: Return Current Disk

Entry Parameters:	
Register C:	25 = 19H
Returned Value:	
Register A:	Current Disk. 0 for A, ... 15 for P
Versions supported:	All versions

Function 25 returns the currently selected default disk number in register A. The disk numbers range from 0 through 15 corresponding to drives A through P.

2.31. Set DMA Address

Function 26: Set DMA Address

Entry Parameters:	
Register C:	26 = 1AH
Registers DE:	DMA Address
Returned Value:	**none**
Versions supported:	All versions

DMA is an acronym for Direct Memory Address, which is often used in connection with disk controllers that directly access the memory of the mainframe computer to transfer data to and from the disk subsystem. Although many computer systems use non-DMA access (that is, the data is transferred through programmed I/O operations), the DMA address has, in CP/M, come to mean the address at which the 128-byte data record resides before a disk write and after a disk read. Upon cold start, warm start, or disk system reset, the DMA address is automatically set to BOOT+0080H. The **Set DMA** function can be used to change this default value to address another area of memory where the data records reside. Thus, the DMA address becomes the value specified by DE until it is changed by a subsequent **Set DMA** function, cold start, warm start, or disk system reset.

2.32. Get Addr(ALLOC)

Function 27: Get Addr(ALLOC)

Entry Parameters:	
Register C:	27 = 1BH
Returned Value:	
Registers HL:	ALLOC Address
Versions supported:	All versions

An allocation vector (ALLOC) is maintained in main memory for each on-line disk drive. Various system programs use the information provided by the allocation vector to determine the amount of remaining storage (see the STAT program). Function 27 returns the base address of the allocation vector for the currently selected disk drive. However, the allocation information might be invalid if the selected disk has been marked Read-Only. Although this function is not normally used by application programs, additional details of the allocation vector are found in Section 6.

2.33. Write Protect Disk

Function 28: Write Protect Disk

Entry Parameters:	
Register C:	28 = 1CH
Returned Value:	**none**
Versions supported:	All versions

The Write Protect Disk function provides temporary write protection for the currently selected disk. Any attempt to write to the disk before the next cold or warm start operation produces the message:

BDOS Err On *d*:R/O

2.34. Get R/O Vector

Function 29: Get R/O Vector

Entry Parameters:	
Register C:	29 = 1DH
Returned Value:	
Registers HL:	R/O Vector Value
Versions supported:	All versions

Function 29 returns a bit vector in register pair HL, which indicates drives that have the temporary Read-Only bit set. As in Function 24, the least significant bit corresponds to drive A, while the most significant bit corresponds to drive P. The R/O bit is set either by an explicit call

to Function 28 or by the automatic software mechanisms within CP/M that detect changed disks.

2.35. Set Echo Mode

Function 30: Set Echo Mode

Entry Parameters:	
Register C:	30 = 1EH
Register E:	0 for no echo, otherwise echo
Returned Value:	**none**
Versions supported:	CP/M 1975 version only

This function is used to alter the behavior of function 1. When the echo flag is set, characters input will be echoed to the console.

2.36. Set File Attributes

Function 30: Set File Attributes

Entry Parameters:	
Register C:	30 = 1EH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Versions supported:	CP/M 2 and later

The Set File Attributes function allows programmatic manipulation of permanent indicators attached to files. In particular, the R/O and System attributes (t1' and t2') can be set or reset. The DE pair addresses an unambiguous filename with the appropriate attributes set or reset. Function 30 searches for a match and changes the matched directory entry to contain the selected indicators. Indicators f1' through f4' are not currently used, but may be useful for applications programs, since they are not involved in the matching process during file open and close operations. Indicators f5' through f8' and t3' are reserved for future system expansion.

2.37. Get Addr(Disk Parameter Block)

Function 31: Get Addr(Disk Parameter Block)

Entry Parameters:	
Register C:	31 = 1FH
Returned Value:	
Registers HL:	**DPB** Address
Versions supported:	CP/M 2 and later

The address of the BIOS resident disk parameter block (**DPB**) is returned in HL as a result of this function call. This address can be used for either of two purposes. First, the disk parameter values can be extracted for display and space computation purposes, or transient programs can dynamically change the values of current disk parameters when the disk environment changes, if required. Normally, application programs will not require this facility.

2.38. Set/Get User Code

Function 32: Set/Get User Code

Entry Parameters:	
Register C:	32 = 20H
Register E:	0FFH (get) or *User Code* (set)
Returned Value:	
Register A:	Current code (get) or none (set)
Versions supported:	CP/M 2 and later

An application program can change or interrogate the currently active user number by calling Function 32. If register E = 0FFH, the value of the current user number is returned in register A, where the value is in the range of 0 to 15. If register E is not 0FFH, the current user number is changed to the value of E, modulo 16.

2.39. Read Random

Function 33: Read Random

Entry Parameters:	
Register C:	33 = 21H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)
Register H:	Hardware error code
Versions supported:	CP/M 2 and later

The **Read Random** function is similar to the sequential file read operation of previous releases, except that the read operation takes place at a particular record number, selected by the 24-bit value constructed from the 3-byte field following the FCB (byte positions **R0** at 33, **R1** at 34, and **R3** at 35). The user should note that the sequence of 24 bits is stored with least significant byte first (**R0**), middle byte next (**R1**), and high byte last (**R2**). CP/M does not reference byte **R2**, except in computing the size of a file (Function 35). Byte **R2** must be zero, however, since a nonzero value indicates overflow past the end of file.

Thus, the **R0**, **R1** byte pair is treated as a double-byte, or word value, that contains the record to read. This value ranges from 0 to 65535, providing access to any particular record of the 8-

megabyte file. To process a file using random access, the base extent (extent 0) must first be opened. Although the base extent might or might not contain any allocated data, this ensures that the file is properly recorded in the directory and is visible in DIR requests. The selected record number is then stored in the random record field (**R0**, **R1**), and the BDOS is called to read the record.

Upon return from the call, register A either contains an error code, as listed below, or the value 00, indicating the operation was successful. In the latter case, the current DMA address contains the randomly accessed record. Note that contrary to the sequential read operation, the record number is not advanced. Thus, subsequent random read operations continue to read the same record.

Upon each random read operation, the logical extent and current record values are automatically set. Thus, the file can be sequentially read or written, starting from the current randomly accessed position. However, note that, in this case, the last randomly read record will be reread as one switches from random mode to sequential read and the last record will be rewritten as one switches to a sequential write operation. The user can simply advance the random record position following each random read or write to obtain the effect of sequential I/O operation.

Error codes returned in register A following a random read are listed below.

01 reading unwritten data

02 (not returned in random mode)

03 cannot close current extent

04 seek to unwritten extent

05 (not returned in read mode)

06 seek past physical end of disk

09 invalid FCB

10 media changed

0FFH hardware error (on 3.x)

Error codes 01 and 04 occur when a random read operation accesses a data block that has not been previously written or an extent that has not been created, which are equivalent conditions. Error code 03 does not normally occur under proper system operation. If it does, it can be cleared by simply rereading or reopening extent zero as long as the disk is not physically write protected. Error code 06 occurs whenever byte **R2** is nonzero under the current 2.0 release. Normally, nonzero return codes can be treated as missing data, with zero return codes indicating operation complete.

2.40. Write Random

Function 34: Write Random

Entry Parameters:	
Register C:	34 = 22H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)
Register H:	Hardware error code
Versions supported:	CP/M 2 and later

The **Write Random** operation is initiated similarly to the Read Random call, except that data is written to the disk from the current DMA address. Further, if the disk extent or data block that is the target of the write has not yet been allocated, the allocation is performed before the write operation continues. As in the Read Random operation, the random record number is not changed as a result of the write. The logical extent number and current record positions of the FCB are set to correspond to the random record that is being written. Again, sequential read or write operations can begin following a random write, with the notation that the currently addressed record is either read or rewritten again as the sequential operation begins. You can also simply advance the random record position following each write to get the effect of a sequential write operation. Note that reading or writing the last record of an extent in random mode does not cause an automatic extent switch as it does in sequential mode.

The error codes returned by a random write are identical to the random read operation with the addition of error code 05, which indicates that a new extent cannot be created as a result of directory overflow.

2.41. Compute File Size

Function 35: Compute File Size

Entry Parameters:	
Register C:	35 = 23H
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB
Versions supported:	CP/M 2 and later

When computing the size of a file, the DE register pair addresses an FCB in random mode format (bytes **R0**, **R1**, and **R2** are present). The FCB contains an unambiguous filename that is used in the directory scan. Upon return, the random record bytes contain the virtual file size, which is, in effect, the record address of the record following the end of the file. Following a call to Function 35, if the high record byte **R2** is 01, the file contains the maximum record count 65536.

Otherwise, bytes **R0** and **R1** constitute a 16-bit value as before (**R0** is the least significant byte), which is the file size.

Data can be appended to the end of an existing file by simply calling Function 35 to set the random record position to the end of file and then performing a sequence of random writes starting at the preset record address.

The virtual size of a file corresponds to the physical size when the file is written sequentially. If the file was created in random mode and holes exist in the allocation, the file might contain fewer records than the size indicates. For example, if only the last record of an 8-megabyte file is written in random mode (that is, record number `65535`), the virtual size is `65536` records, although only one block of data is actually allocated.

2.42. Set Random Record

Function 36: Set Random Record

Entry Parameters:	
Register C:	`36 = 24H`
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB
Versions supported:	CP/M 2 and later

The Set Random Record function causes the BDOS automatically to produce the random record position from a file that has been read or written sequentially to a particular point. The function can be useful in two ways.

First, it is often necessary initially to read and scan a sequential file to extract the positions of various key fields. As each key is encountered, Function 36 is called to compute the random record position for the data corresponding to this key. If the data unit size is 128 bytes, the resulting record position is placed into a table with the key for later retrieval. After scanning the entire file and tabulating the keys and their record numbers, the user can move instantly to a particular keyed record by performing a random read, using the corresponding random record number that was saved earlier. The scheme is easily generalized for variable record lengths, because the program need only store the buffer-relative byte position along with the key and record number to find the exact starting position of the keyed data at a later time.

A second use of Function 36 occurs when switching from a sequential read or write over to random read or write. A file is sequentially accessed to a particular point in the file, Function 36 is called, which sets the record number, and subsequent random read and write operations continue from the selected point in the file.

2.43. Reset Drives

Function 37: Reset Drives

Entry Parameters:	
Register C:	37 = 25H
Registers DE:	Drive Vector
Returned Value:	
Register A:	00H
Versions supported:	CP/M 2.2 and later

The Reset Drive function allows resetting of specified drives. The passed parameter is a 16-bit vector of drives to be reset; the least significant bit is drive A.

To maintain compatibility with MP/M, CP/M returns a zero value.

2.44. Access Drive

Function 38: Access Drive

Entry Parameters:	
Register C:	38 = 26H
Returned Value:	
Register A:	00H
Versions supported:	CP/M 2.2 and later

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero In register A indicating that the access request is successful.

2.45. Free Drive

Function 39: Free Drive

Entry Parameters:	
Register C:	39 = 27H
Returned Value:	
Register A:	00H
Versions supported:	CP/M 2.2 and later

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero In register A indicating that the free request is successful.

2.46. Write Random with Zero Fill

Function 40: Write Random with Zero Fill

Entry Parameters:	
Register C:	40 = 28H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code

Register H:	Hardware Error
Versions supported:	CP/M 2.2 and later

The Write With Zero Fill operation is similar to Function 34, with the exception that a previously unallocated block is filled with zeros before the data is written.

2.47. Test and Write Record

Function 41: Test and Write Record

Entry Parameters:	
Register C:	41 = 29H
Registers DE:	FCB Address
Returned Value:	
Register A:	Error Code (0FFH)
Register H:	Hardware Error Code (0)
Versions supported:	CP/M 3 only

The Test and Write Record function is an MP/M II function that is not supported under CP/M 3. If called, Function 41 returns with register A set to 0FFH and register H set to zero.

2.48. Lock Record

Function 42: Lock Record

Entry Parameters:	
Register C:	42 = 2AH
Registers DE:	FCB Address
Returned Value:	
Register A:	Error Code (00H)
Versions supported:	CP/M 3 only

The Lock Record function is an MP/M II function that is supported under CP/M 3 only to provide compatibility between CP/M 3 and MP/M. It is intended for use in situations where more than one running program has Read-Write access to a common file. Because CP/M 3 is a single-user operating system in which only one program can run at a time, this situation cannot occur. Thus, under CP/M 3, Function 42 performs no action except to return the value 00H in register A indicating that the record lock operation is successful.

2.49. Unlock Record

Function 43: Unlock Record

Entry Parameters:	
Register C:	43 = 2BH
Registers DE:	FCB Address

Returned Value:	
Register A:	00H
Versions supported:	CP/M 3 only

The Unlock Record function is an MP/M II function that is supported under CP/M 3 only to provide compatibility between CP/M 3 and MP/M. It is intended for use in situations where more than one running program has Read-Write access to a common file. Because CP/M 3 is a single-user operating system in which only one program can run at a time, this situation cannot occur. Thus, under CP/M 3, Function 43 performs no action except to return the value 00H in register A indicating that the record unlock operation is successful.

2.50. Set Multi-Sector Count

Function 44: Set Multi-Sector Count

Entry Parameters:	
Register C:	44 = 2CH
Register E:	Number of Sectors
Returned Value:	
Register A:	Return Code
Versions supported:	CP/M 3 only

The Set Multi-Sector Count function provides logical record blocking under CP/M 3. It enables a program to read and write from 1 to 128 records of 128 bytes at a time during subsequent BDOS Read and Write functions.

Function 44 sets the Multi-Sector Count value for the calling program to the value passed in register E. Once set, the specified Multi-Sector Count remains in effect until the calling program makes another Set Multi-Sector Count function call and changes the value. Note that the CCP sets the Multi-Sector Count to one (1) when it initiates a transient program.

The Multi-Sector Count affects BDOS error reporting for the BDOS Read and Write functions. If an error interrupts these functions when the Multi-Sector is greater than one, they return the number of records successfully read or written in register H for all errors except for physical errors (A = 255 = 0FFH).

Upon return, register A is set to zero (0) if the specified value is in the range of 1 to 128. Otherwise, register A is set to 0FFH.

2.51. Set BDOS Error Mode

Function 45: Set BDOS Error Mode

Entry Parameters:	
Register C:	45 = 2DH

Register E:	BDOS Error Mode
Returned Value:	**none**
Versions supported:	CP/M 3 only

Function 45 sets the BDOS error mode for the calling program to the mode specified in register E. If register E is set to 0FFH, 255 decimal, the error mode is set to Return Error mode. If register E is set to 0FEH, 254 decimal, the error mode is set to Return and Display mode. If register E is set to any other value, the error mode is set to the default mode.

The SET BDOS Error Mode function determines how physical and extended errors are handled for a program. The Error Mode can exist in three modes: the **default** mode, **Return Error** mode, and **Return and Display Error** mode. In the **default** mode, the BDOS displays a system message at the console that identifies the error and terminates the calling program. In the **Return Error** and **Display and Return Error** modes, the BDOS sets register A to 0FFH, 255 decimal, places an error code that identifies the physical or extended error in register H and returns to the calling program. In **Return and Display** mode, the BDOS displays the system message before returning to the calling program. No system messages are displayed, however, when the BDOS is in **Return Error** mode.

2.52. Get Disk Free Space

Function 46: Get Disk Free Space

Entry Parameters:	
Register C:	46 = 2EH
Register E:	Drive ID
Returned Value:	
Register A:	Error Flag
Register H:	Hardware Error
Other:	First 3 bytes of DMA buffer
Versions supported:	CP/M 3 only

The Get Disk Free Space function determines the number of free sectors, 128 byte records, on the specified drive. The calling program passes the drive number in register E, with 0 for drive A, 1 for B, and so on, through 15 for drive P in a full 16-drive system. Function 46 returns a binary number in the first 3 bytes of the current DMA buffer. This number is returned in the following format:

+0 low byte

+1 middle byte

+2 high byte

Note that the returned free space value might be inaccurate if the drive has been marked Read-Only.

Upon return, register A is set to zero if the function is successful. However, if the BDOS Error Mode is one of the return modes (see Function 45), and a physical error is encountered, register A is set to 0FFH, 255 decimal, and register H is set to one of the following values:

01 Disk I/O error

04 Invalid drive error

2.53. Chain to Program

Function 47: Chain to Program

Entry Parameters:	
Register C:	47 = 2FH
Register E:	Chain Flag
Returned Value:	**Does not return** to calling program
Versions supported:	CP/M 3 only

The Chain To Program function provides a means of chaining from one program to the next without operator intervention. The calling program must place a command line terminated by a null byte, 00H, in the default DMA buffer. If register E is set to 0FFH, the CCP initializes the default drive and user number to the current program values when it passes control to the specified transient program. Otherwise, these parameters are set to the default CCP values. Note that Function 108, Get/Set Program Return Code, can be used to pass a two byte value to the chained program.

Function 47 does not return any values to the calling program and any encountered errors are handled by the CCP.

2.54. Flush Buffers

Function 48: Flush Buffers

Entry Parameters:	
Register C:	48 = 30H
Register E:	Purge Flag
Returned Value:	
Register A:	Error Flag
Register H:	Hardware Error Flag
Versions supported:	CP/M 3 only

The Flush Buffers function forces the write of any write-pending records contained in internal blocking/deblocking buffers. If register E is set to 0FFH, this function also purges all active data buffers. Programs that provide write with read verify support need to purge internal buffers to

ensure that verifying reads actually access the disk instead of returning data that is resident in internal data buffers. The CP/M 3 PIP utility is an example of such a program.

Upon return, register A is set to zero if the flush operation is successful. If a physical error is encountered, the Flush Buffers function performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is in the default mode, a message identifying the error is displayed at the console and the calling program is terminated. Otherwise, the Flush Buffers function returns to the calling program with register A set to 0FFH and register H set to the following physical error code:

01 Disk I/O error

02 Read/only disk

04 Invalid drive error

2.55. Set/Get System Control Block

Function 49: Set/Get System Control Block

Entry Parameters:	
Register C:	49 = 31H
Registers DE:	SCB PB Address
Returned Value:	
Register A:	returned byte
Registers HL:	returned word
Versions supported:	CP/M 3 only

Function 49 allows access to parameters located in the CP/M 3 System Control Block (SCB). The SCB is a 100-byte data structure residing within the BDOS that contains flags and data used by the BDOS, CCP and other system components. Note that Function 49 is a CP/M 3 specific function. Programs intended for both MP/M II and CP/M 3 should either avoid the use of this function or isolate calls to this function in CP/M 3 version-dependent sections. To use Function 49, the calling program passes the address of a data structure called the SCB parameter block in register pair DE. This data structure identifies the byte or word of the SCB to be updated or returned. The SCB parameter block is defined as:

```
SCBPB:
          DB OFFSET     ;   Offset within SCB
          DB SET        ; `0FFH` if setting a byte
                        ; `0FEH` if setting a word
                        ; `01H` - `0FDH` are reserved
                        ; `00H` if a get operation
          DW VALUE      ; Byte or word value to be set
```

The OFFSET parameter identifies the offset of the field within the SCB to be updated or accessed. The SET parameter determines whether Function 49 is to set a byte or word value in the SCB or if it is to return a byte from the SCB. The VALUE parameter is used only in set calls. In addition, only the first byte of VALUE is referenced in set byte calls.

If Function 49 is called with the OFFSET parameter of the SCB parameter block greater than 63H, the function performs no action but returns with registers A and HL set to zero.

Use caution when you set SCB fields. Some of these parameters reflect the current state of the operating system. If they are set to invalid values, software errors can result. In general, do not use Function 49 to set a system parameter if another BDOS function can achieve the same result. For example, Function 49 can be called to update the Current DMA Address field within the SCB. This is not equivalent to making a Function 26, Set DMA Address call, and updating the SCB Current DMA field in this way would result in system errors. However, you can use Function 49 to return the Current DMA address.

The System Control Block is summarized in the following table. Each of these fields is documented in detail in Appendix A.

2.56. Direct BIOS Calls

Function 50: Direct BIOS Calls

Entry Parameters:	
Register C:	50 = 32H
Registers DE:	BIOS PB Address
Returned Value:	BIOS Return Value
Versions supported:	CP/M 3 only

Function 50 provides a direct BIOS call through the BDOS to the BIOS. The calling program passes the address of a data structure called the BIOS Parameter Block (BIOSPB) in register pair DE. The BIOSPB contains the BIOS function number and register contents as shown below:

```
BIOSPB: DB  FUNC        ; BIOS function no.
        DB  AREG        ; A register contents
        DW  BCREG       ; BC register contents
        DW  DEREG       ; DE register contents
        DW  HLREG       ; HL register contents
```

System Reset (Function 0) is equivalent to Function 50 with a BIOS function number of 1. Note that the register pair BIOSPB fields (BCREG, DEREG, HLREG) arc defined in low byte, high byte order. For example, in the BCREG field, the first byte contains the C register value, the second byte contains the B register value. Under CP/M 3, direct BIOS calls via the BIOS jump vector are only supported for the BIOS Console I/O and List functions. You must use Function 50 to call any other BIOS functions. In addition, Function 50 intercepts BIOS Function 27 (Select Memory)

calls and returns with register A set to zero. Refer to the CP/M Plus (CP/M Version 3) Operating System System Guide for the definition of the BIOS functions and their register passing and return conventions.

2.57. Load Overlay

Function 59: Load Overlay

Entry Parameters:	
Register C:	59 = 3BH
Registers DE:	FCB Address
Returned Value:	
Register A:	Error Code
Register H:	Hardware Error Code
Versions supported:	CP/M 3 only

Only transient programs with an RSX header can use the Load Overlay function because BDOS Function 59 is supported by the LOADER module. The calling program must have a header to force the LOADER to remain resident after the program is loaded (see Section 1.3).

Function 59 loads either an absolute or relocatable module. Relocatable modules are identified by a filetype of PRL. Function 59 does not call the loaded module.

The referenced FCB must be successfully opened before Function 59 is called. The load address is specified in the first two random record bytes of the FCB, **RO** and **R1**. The LOADER returns an error if the load address is less than 0100H, or if performing the requested load operation would overlay the LOADER, or any other Resident System Extensions that have been previously loaded.

When loading relocatable files, the LOADER requires enough room at the load address for the complete PRL file including the header and bit map (see Appendix B). Otherwise an error is returned. Function 59 also returns an error on PRL file load requests if the specified load address is not on a page boundary.

Upon return, Function 59 sets register A to zero if the load operation is successful. If the LOADER RSX is not resident in memory because the calling program did not have a RSX header, the BDOS returns with register A set to 0FFH and register H set to zero. If the LOADER detects an invalid load address, or if insufficient memory is available to load the overlay, Function 59 returns with register A set to 0FEH. All other error returns are consistent with the error codes returned by BDOS Function 20, Read Sequential.

2.58. Call Resident System Extension

Function 60: Call Resident System Extension

Entry Parameters:	
Register C:	60 = 3CH

Registers DE:	RSX PB Address
Returned Value:	
Register A:	Error Code
Register H:	Hardware Error Code
Versions supported:	CP/M 3 only

Function 60 is a special BDOS function that you use when you call Resident System Extensions. The RSX sub-function is specified in a structure called the RSX Parameter Block, defined as follows:

```
RSXPB:  DB  FUNC      ; RSX Function number
        DB  NUMPARMS  ; Number of word Parameters
        DW  PARMETER1 ; Parameter 1
        DW  PARMETER2 ; Parameter 2
        ...
        DW  PARMETERN ; Parameter n
```

RSX modules filter all BDOS calls and capture RSX function calls that they can handle. If there is no RSX module present in memory that can handle a specific RSX function call, the call is not trapped, and the BDOS returns 0FFH in registers A and L. RSX function numbers from 0 to 127 are available for CP/M 3 compatible software use. RSX function numbers 128 to 255 are reserved for system use.

2.59. Free Blocks

Function 98: Free Blocks

Entry Parameters:	
Register C:	98 = 62H
Returned Value:	
Register A:	Error Flag
Register H:	Hardware Error Flag
Versions supported:	CP/M 3 only

The Free Blocks function scans all the currently logged-in drives, and for each drive returns to free space all temporarily-allocated data blocks. A temporarily allocated data block is a block that has been allocated to a file by a BDOS write operation but has not been permanently recorded in the directory by a BDOS close operation. The CCP calls Function 98 when it receives control following a system warm start. Be sure to close your file, particularly any file you have written to, prior to calling Function 98.

In the nonbanked version of CP/M 3, Function 98 frees only temporarily allocated blocks for systems that request double allocation vectors in GENCPM.

Upon return, register A is set to zero if Function 98 is successful. If a physical error is encountered, the Free Blocks function performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is in the default mode, a message identifying the error is displayed at the console and the calling program is terminated. Otherwise, the Free Blocks function returns to the calling program with register A set to 0FFH and register H set to the following physical error code:

04 Invalid drive error

2.60. Truncate File

Function 99: Truncate File

Entry Parameters:	
Register C:	99 = 63H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Register H:	Hardware Error Code
Versions supported:	CP/M 3 only

The Truncate File function sets the last record of a file to the random record number contained in the referenced FCB. The calling program passes the address of the FCB in register pair DE, with byte 0 of the FCB specifying the drive, bytes 1 through 11 specifying the filename and filetype, and bytes 33 through 35, rO, rl, and r2, specifying the last record number of the file. The last record number is a 24 bit value, stored with the least significant byte first, rO, the middle byte next, rl, and the high byte last, r2. This value can range from 0 to 262,143, which corresponds to a maximum value of 3 in byte r2.

If the file specified by the referenced FCB is password protected, the correct pass- word must be placed in the first eight bytes of the current DMA buffer, or have been previously established as the default password (see Function 106).

Function 99 requires that the file specified by the FCB not be open, particularly if the file has been written to. In addition, any activated FCBs naming the file are not valid after Function 99 is called. Close your file before calling Function 99, and then reopen it after the call to continue processing on the file.

Function 99 also requires that the random record number field of the referenced FCB specify a value less than the current file size. In addition, if the file is sparse, the random record field must specify a record in a region of the file where data exists.

Upon return, the Truncate function returns a Directory Code in register A with the value 0 if the Truncate function is successful, or 0FFH, 255 decimal, if the file is not found or the record

number is invalid. Register H is set to zero in both of these cases. If a physical or extended error is encountered, the Truncate function performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is in the default mode, a message identifying the error is displayed at the console and the program is terminated. Otherwise, the Truncate function returns to the calling program with register A set to 0FFH and register H set to one of the following physical or extended error codes:

01 Disk I/O error

02 Read-Only disk

03 Read-Only file

04 Invalid drive error

07 File password error

09 ? in filename or filetype field

2.61. Set Directory Label

Function 100: Set Directory Label

Entry Parameters:	
Register C:	100 = 64H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Register H:	Hardware Error Code
Versions supported:	CP/M 3 only

The Set Directory Label function creates a directory label, or updates the existing directory label for the specified drive. The calling program passes in register pair DE the address of an FCB containing the name, type, and extent fields to be assigned to the directory label. The name and type fields of the referenced FCB are not used to locate the directory label in the directory; they are simply copied into the updated or created directory label. The extent field of the FCB, byte 12, contains the user's specification of the directory label data byte. The definition of the directory label data byte:

bit 7 Require passwords for password-protected files (Not supported in nonbanked CP/M 3 systems)

bit 6 Perform access date and time stamping

bit 5 Perform update date and time stamping

bit 4 Perform create date and time stamping

bit 0 Assign a new password to the directory label

If the current directory label is password protected, the correct password must be placed in the first eight bytes of the current DMA, or have been previously established as the default password (see Function 106). If bit 0, the low-order bit, of byte 12 of the FCB is set to 1, it indicates that a new password for the directory label has been placed in the second eight bytes of the current DMA.

Note that Function 100 is implemented as an RSX, DIRLBL.RSX, in nonbanked CP/M 3 systems. If Function 100 is called in nonbanked systems when the DIRLBL.RSX is not resident an error code of 0FFH is returned.

Function 100 also requires that the referenced directory contain SFCBs to activate date and time stamping on the drive. If an attempt is made to activate date and time stamping when no SFCBs exist, Function 100 returns an error code of 0FFH in register A and performs no action. The CP/M 3 INITDIR utility initializes a directory for date and time stamping by placing an SFCB record in every fourth entry of the directory.

Function 100 returns a Directory Code in register A with the value 0 if the directory label create or update is successful, or 0FFH, 255 decimal, if no space exists in the referenced directory to create a directory label, or if date and time stamping was requested and the referenced directory did not contain SFCBS. Register H is set to zero in both of these cases. If a physical error or extended error is encountered, Function 100 performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is the default mode, a message identifying the error is displayed at the console and the calling program is terminated. Otherwise, Function 100 returns to the calling program with register A set to 0FFH and register H set to one of the following physical or extended error codes:

01 Disk I/O error

02 Read-only disk

04 Invalid drive error

07 File password error

2.62. Return Directory Label Data

Function 101: Return Directory Label Data

Entry Parameters:	
Register C:	101 = 65H
Register E:	Drive ID
Returned Value:	
Register A:	Directory Label Data Byte
Register H:	Hardware Error Code
Versions supported:	CP/M 3 only

The Return Directory Label Data function returns the data byte of the directory label for the specified drive. The calling program passes the drive number in register E with 0 for drive A, 1 for drive B, and so on through 15 for drive P in a full sixteen drive system. The format of the directory label data byte is shown below:

bit 7 Require passwords for password protected files

bit 6 Perform access date and time stamping

bit 5 Perform update date and time stamping

bit 4 Perform create date and time stamping

bit 0 Directory label exists on drive

Function 101 returns the directory label data byte to the calling program in register A. Register A equal to zero indicates that no directory label exists on the specified drive. If a physical error is encountered by Function 101 when the BDOS Error mode is in one of the return modes (see Function 45), this function returns with register A set to 0FFH, 25S decimal, and register H set to one of the following:

01 Disk I/O error

04 Invalid drive error

2.63. Read File Date Stamps and Password Mode

Function 102: Read File Date Stamps and Password Mode

Entry Parameters:	
Register C:	102 = 66H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Register H:	Hardware Error Code
Other:	fields in FCB are updated
Versions supported:	CP/M 3 only

Function 102 returns the date and time stamp information and password mode for the specified file in byte 12 and bytes 24 through 32 of the specified FCB. The calling program passes in register pair DE, the address of an FCB in which the drive, file- name, and filetype fields have been defined.

If Function 102 is successful, it sets the following fields in the referenced FCB:

- byte 12 : Password mode field

  bit 7 Read mode

  bit 6 Write mode

  bit 4 Delete mode

Byte 12 equal to zero indicates the file has not been assigned a password. In nonbanked systems, byte 12 is always set to zero.

- byte 24 - 27 Create or Access time stamp field
- byte 28 - 31 Update time stamp field

The date stamp fields are set to binary zeros if a stamp has not been made. The format of the time stamp fields is the same as the format of the date and time structure described in Function 104.

Upon return, Function 102 returns a Directory Code in register A with the value zero if the function is successful, or 0FFH, 255 decimal, if the specified file is not found. Register H is set to zero in both of these cases. If a physical or extended error is encountered, Function 102 performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is in the default mode, a message identifying the error is displayed at the console and the calling program is terminated. Otherwise, Function 102 returns to the calling program with register A set to 0FFH and register H set to one of the following physical or extended error codes:

01 Disk I/O error

04 Invalid drive error

09 ? in filename or filetype field

2.64. Write File XFCB

Function 103: Write File XFCB

Entry Parameters:	
Register C:	103 = 67H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code
Register H:	Hardware Error Code
Other:	XFCB fields updated
Versions supported:	CP/M 3 only

The Write File XFCB function creates a new XFCB or updates the existing XFCB for the specified file. The calling program passes in register pair DE the address of an FCB in which the drive, name, type, and extent fields have been defined. The extent field specifies the password mode and whether a new password is to be assigned to the file. The format of the extent byte is shown below:

- FCB byte 12 (ex) : XFCB password mode

bit 7 Read mode

bit 6 Write mode

bit 5 Delete mode

bit 0 Assign new password to the file

If the specified file is currently password protected, the correct password must reside in the first eight bytes of the current DMA, or have been previously established as the default password (see Function 106). If bit 0 is set to 1, the new password must reside in the second eight bytes of the current DMA.

Upon return, Function 103 returns a Directory Code in register A with the value zero if the XFCB create or update is successful, or 0FFH, 255 decimal, if no directory label exists on the specified drive, or the file named in the FCB is not found, or no space exists in the directory to create an XFCB. Function 103 also returns with 0FFH in register A if passwords are not enabled by the referenced directory's label. On nonbanked systems, this function always returns with register A = 0FFH because passwords are not supported. Register H is set to zero in all of these cases. If a physical or extended error is encountered, Function 103 performs different actions depending on the BDOS error mode (see Function 45). If the BDOS error mode is the default mode, a message identifying the error is displayed at the console and the calling program is terminated. Otherwise, Function 103 returns to the calling program with register A set to 0FFH and register H set to one of the following physical or extended error codes:

01 Disk I/O error

02 Read-Only disk

04 Invalid drive error

07 File password error

09 ? in filename or filetype field

2.65. Set Date and Time

Function 104: Set Date and Time

Entry Parameters:	
Register C:	104 = 68H
Registers DE:	DAT Address
Returned Value:	**none**
Versions supported:	CP/M 3 only

The Set Date and Time function sets the system internal date and time. The calling program passes the address of a 4-byte structure containing the date and time specification in the register pair DE. The format of the date and time (DAT) data structure is:

```
DAT:    DW    DATE        ; Days since January 1, 1978
        DB    HOUR        ; Hour field (2-BCD digits)
        DB    MINUTE      ; Minute field (2-BCD digits)
```

The date is represented as a 16-bit integer with day 1 corresponding to January 1, 1978. The time is represented as two bytes: hours and minutes are stored as two BCD digits. This function also sets the seconds field of the system date and time to zero.

2.66. Get Date and Time

Function 105: Get Date and Time

Entry Parameters:	
Register C:	105 = 69H
Registers DE:	DAT Address
Returned Value:	
Register A:	seconds (two-digit BCD value)
Other:	DAT structure is updated
Versions supported:	CP/M 3 only

The Get Date and Time function obtains the system internal date and time. The calling program passes in register pair DE, the address of a 4-byte data structure which receives the date and time values. The format of the date and time, DAT, data structure is the same as the format described in Function 104. Function 105 also returns the seconds field of the system date and time in register A as a two digit BCD value.

The format of the date and time (DAT) data structure is:

```
DAT:    DW    DATE        ; Days since January 1, 1978
        DB    HOUR        ; Hour field (2-BCD digits)
        DB    MINUTE      ; Minute field (2-BCD digits)
```

2.67. Set Default Password

Function 106: Set Default Password

Entry Parameters:	
Register C:	106 = 6AH
Registers DE:	Password Address
Returned Value:	**none**
Versions supported:	CP/M 3 banked only

The Set Default Password function allows a program to specify a password value before a file protected by the password is accessed. When the file system accesses a password-protected file, it checks the current DMA, and the default password for the correct value. If either value matches the file's password, full access to the file is allowed. Note that this function performs no action in nonbanked CP/M 3 systems because file passwords are not supported.

To make a Function 106 call, the calling program sets register pair DE to the address of an 8-byte field containing the password.

2.68. Return Serial Number

Function 107: Return Serial Number

Entry Parameters:	
Register C:	107 = 6BH
Registers DE:	Serial Number Field
Returned Value:	Serial Number Field is set
Versions supported:	CP/M 3 only

Function 107 returns the CP/M 3 serial number to the 6-byte field addressed by register pair DE.

2.69. Set/Get Program Return Code

Function 108: Set/Get Program Return Code

Entry Parameters:	
Register C:	108 = 6CH
Registers DE:	0FFFFH (Get) or Program Return Code (Set)
Returned Value:	
Registers HL:	Program Return Code or (none)
Versions supported:	CP/M 3 only

CP/M 3 allows programs to set a return code before terminating. This provides a mechanism for programs to pass an error code or value to a following job step in batch environments. For example, Program Return Codes are used by the CCP in CP/M 3's conditional command line batch facility. Conditional command lines are command lines that begin with a colon, :. The execution of a conditional command depends on the successful execution of the preceding command. The CCP tests the return code of a terminating program to determine whether it successfully completed or terminated in error. Program return codes can also be used by programs to pass an error code or value to a chained program (see Function 47, Chain To Program).

A program can set or interrogate the Program Return Code by calling Function 108. If rester pair DE = 0FFFFH, then the current Program Return Code is returned in register pair HL. Otherwise, Function 108 sets the Program Return Code to the value contained in register pair DE. Program Return Codes are defined in the table below.

Code	Meaning
0000 ... FEFF	Successful Return
FF00 ... FFFE	Unsuccessful Return

Code	Meaning
0000	The CCP initializes the Program Return Code to zero unless the program is loaded as the result of program chain.
FF80 ... FFFC	Reserved
FFFD	The program is terminated because of a fatal BDOS error.
FFFE	The program is terminated by the BDOS because the user typed a CTRL-C.

2.70. Set/Get Console Mode

Function 109: Set/Get Console Mode

Entry Parameters:	
Register C:	109 = 6DH
Registers DE:	0FFFFH (Get) or Console Mode (Set)
Returned Value:	
Registers HL:	Console Mode or (none)
Versions supported:	CP/M 3 only

A program can set or interrogate the Console Mode by calling Function 109. If register pair DE = 0FFFFH, then the current Console Mode is returned in register HL. Otherwise, Function 109 sets the Console Mode to the value contained in register pair DE. The Console Mode is a 16-bit system parameter that determines the action of certain BDOS Console I/O functions. The definition of the Console Mode is:

- bit 0 =
  - **0** Normal status for Function 11.
  - **1** CTRL-C only status for Function 11.
- bit 1 =
  - **0** Enable Stop Scroll, Start Scroll supported.
  - **1** Disable Stop Scroll, CTRL-S, Start Scroll, CTRL-Q Support.
- bit 2 =
  - **0** Normal console output mode.
  - **1** Raw console output mode. Disables tab expansion for Functions 2, 9 and 111. Also disables printer echo, CTRL-P support.
- bit 3 =
  - **0** Enable CTRL-C program termination.
  - **1** Disable CTRL-C program termination.
- bits 8, 9 =

00 conditional status

01 false status

10 true status

11 bypass redirection

Note that the Console Mode bits are numbered from right to left. Bit 0 is the least significant bit.

The CCP initializes the Console Mode to zero when it loads a program unless the program has an RSX that overrides the default value. Refer to Section 2.2.1 for detailed information on Console Mode.

2.71. Set/Get Output Delimiter

Function 110: Set/Get Output Delimiter

Entry Parameters:	
Register C:	110 = 6EH
Register E:	Output Delimiter (Set) or
Registers DE:	0FFFFH (Get)
Returned Value:	
Register A:	Output Delimiter or (none)
Versions supported:	CP/M 3 only

A program can set or interrogate the current Output Delimiter by calling Function 110. If register pair DE = 0FFFFH, then the current Output Delimiter is returned in register A. Otherwise, Function 110 sets the Output Delimiter to the value contained in register E.

Function 110 sets the string delimiter for Function 9, Print String. The default delimiter value is a dollar sign, $. The CCP restores the Output Delimiter to the default value when a transient program is loaded.

2.72. Print Block

Function 111: Print Block

Entry Parameters:	
Register C:	111 = 6FH
Registers DE:	CCB Address
Returned Value:	**none**
Versions supported:	CP/M 3 only

The Print Block function sends the character string located by the Character Control Block, CCB, addressed in register pair DE, to the logical console, CONOUT:. If the Console Mode is in the default state (see Section 2.2.1), Function 111 expands tab characters, CTRL-I, in columns of eight characters. It also checks for stop scroll, CTRL-S, start scroll, CTRL-Q, and echoes to the logical list device, LST:, if printer echo, CTRL-P, has been invoked.

The CCB format is:

```
CCB:    DW    STR         ; Address of ASCII string
        DW    LENGTH      ; Length of character string
```

2.73. List Block

Function 112: List Block

Entry Parameters:	
Register C:	112 = 70H
Registers DE:	CCB Address
Returned Value:	**none**
Versions supported:	CP/M 3 only

The List Block function sends the character string located by the Character Control Block, CCB, addressed in register pair DE, to the logical list device, LST:.

The CCB format is:

```
CCB:    DW    STR         ; Address of ASCII string
        DW    LENGTH      ; Length of character string
```

2.74. Parse Filename

Function 152: Parse Filename

Entry Parameters:	
Register C:	152 = 98H
Registers DE:	PFCB Address
Returned Value:	
Registers HL:	Return Code
Other:	Parsed File Control Block (FCB)
Versions supported:	CP/M 3 only

The Parse Filename function parses an ASCII file specification and prepares a File Control Block, FCB. The calling program passes the address of a data structure called the Parse Filename Control Block, PFCB, in register pair DE. The PFCB contains the address of the input ASCII filename string followed by the address of the target FCB as shown below:

```
PFCB:   DW    INPUT     ; Address of input ASCII string
        DW    FCB       ; Address of target FCB
```

The maximum length of the input ASCII string to be parsed is 128 bytes. The target FCB must be 36 bytes in length.

Function 152 assumes the input string contains file specifications in the following form:

{*d*:}*filename*{.*typ*}{;*password*}

where items enclosed in curly brackets are optional. Function 152 also accepts isolated drive specifications *d*: in the input string. When it encounters one, it sets the filename, filetype, and password fields in the FCB to blank.

The Parse Filename function parses the first file specification it finds in the input string. The function first eliminates leading blanks and tabs. The function then assumes that the file specification ends on the first delimiter it encounters that is out of context with the specific field it is parsing. For instance, if it finds a colon, and it is not the second character of the file specification, the colon delimits the entire file specification.

Function 152 recognizes the following characters as delimiters:

- space
- tab
- return
- CTRL-J
- ; (semicolon) – except before password field.
- = (equal)
- < (less than)
- > (greater than)
- . (period) – except after filename and before filetype
- : (colon) – except before filename and after drive.
- , (comma)
- | (vertical bar)
- [(left square bracket)
-] (right square bracket)

If Function 152 encounters a non-graphic character in the range 1 through 31 not listed above, it treats the character as an error. The Parse Filename function initializes the specified FCB shown in the table below.

Location	Contents
byte 0	The drive field is set to the specified drive. If the drive is not specified, the default drive code is used. 0 = default, 1 = A, 2 = B, ... 15 = P.
bytes 1-8	The name is set to the specified filename. All letters are converted to upper-case. If the name is not eight characters long, the remaining bytes in the filename field are padded with blanks. If the filename has an asterisk, *, all

Location	Contents
	remaining bytes in the filename field are filled in with question marks, ?. An error occurs if the filename is more than eight bytes long.
bytes 9-11	The type is set to the specified filetype. If no filetype is specified, the type field is initialized to blanks. All letters are converted to upper-case. If the type is not three characters long, the remaining bytes in the filetype field are padded with blanks. If an asterisk, *, occurs, all remaining bytes are filled in with question marks, ?. An error occurs if the type field is more than three bytes long.
bytes 12-15	Filled in with zeros
bytes 16-23	The password field is set to the specified password. If no password is specified, it is initialized to blanks. If the password is less than eight characters long, remaining bytes are padded with blanks. All letters are converted to upper-case. If the password field is more than eight bytes long, an error occurs. Note that a blank in the first position of the password field implies no password was specified.
bytes 24-31	Reserved for system use.

If an error occurs, Function 152 returns an 0FFFFH in register pair HL.

On a successful parse, the Parse Filename function checks the next item in the input string. It skips over trailing blanks and tabs and looks at the next character. If the character is a null or carriage return, it returns a 0 indicating the end of the input string. If the character is a delimiter, it returns the address of the delimiter. If the character is not a delimiter, it returns the address of the first trailing blank or tab.

If the first non-blank or non-tab character in the input string is a null, 0, or carriage return, the Parse Filename function returns a zero indicating the end of string.

If the Parse Filename function is to be used to parse a subsequent file specification in the input string, the returned address must be advanced over the delimiter before placing it in the PFCB.

3. System Call Support

		CP/M Version						
#	Function Name	1975	1.3	1.4	2.0	2.2	3.0 non-banked	3.0 banked
0	System Reset	✅	✅	✅	✅	✅	✅	✅
1	Console Input	✅	✅	✅	✅	✅	✅	✅
2	Console Output	✅	✅	✅	✅	✅	✅	✅
3	Reader Input	✅	✅	✅	✅	✅	✅	✅
4	Punch Output	✅	✅	✅	✅	✅	✅	✅
5	List Output	✅	✅	✅	✅	✅	✅	✅
6	Direct Console I/O	🚫	🚫	🚫	✅	✅	✅	✅
6	Memory Size	✅	✅	✅	🚫	🚫	🚫	🚫
7	Auxiliary Input Status	🚫	🚫	🚫	🚫	🚫	✅	✅
7	Get IOBYTE	✅	✅	✅	✅	✅	🚫	🚫
8	Auxiliary Output Status	🚫	🚫	🚫	🚫	🚫	✅	✅
8	Set IOBYTE	✅	✅	✅	✅	✅	🚫	🚫
9	Print String	✅	✅	✅	✅	✅	✅	✅
10	Read Console Buffer	✅	✅	✅	✅	✅	✅	✅
11	Get Console Status	✅	✅	✅	✅	✅	✅	✅
12	Lift Head	✅	✅	✅	🚫	🚫	🚫	🚫
12	Version Number	🚫	🚫	🚫	✅	✅	✅	✅
13	Reset Disk System	✅	✅	✅	✅	✅	✅	✅
14	Select Disk	✅	✅	✅	✅	✅	✅	✅
15	Open File	✅	✅	✅	✅	✅	✅	✅
16	Close File	✅	✅	✅	✅	✅	✅	✅
17	Search for First	✅	✅	✅	✅	✅	✅	✅
18	Search for Next	✅	✅	✅	✅	✅	✅	✅
19	Delete File	✅	✅	✅	✅	✅	✅	✅
20	Read Sequential	✅	✅	✅	✅	✅	✅	✅
21	Write Sequential	✅	✅	✅	✅	✅	✅	✅

		CP/M Version						
#	**Function Name**	**1975**	**1.3**	**1.4**	**2.0**	**2.2**	**3.0 non-banked**	**3.0 banked**
22	Make File	✅	✅	✅	✅	✅	✅	✅
23	Rename File	✅	✅	✅	✅	✅	✅	✅
24	Return Login Vector	✅	✅	✅	✅	✅	✅	✅
25	Return Current Disk	✅	✅	✅	✅	✅	✅	✅
26	Set DMA Address	✅	✅	✅	✅	✅	✅	✅
27	Get Addr(ALLOC)	✅	✅	✅	✅	✅	✅	✅
28	Write Protect Disk	✅	✅	✅	✅	✅	✅	✅
29	Get R/O Vector	✅	✅	✅	✅	✅	✅	✅
30	Set Echo Mode	✅	🚫	🚫	🚫	🚫	🚫	🚫
30	Set File Attributes	🚫	🚫	🚫	✅	✅	✅	✅
31	Get Addr(Disk Parameter Block)	🚫	🚫	🚫	✅	✅	✅	✅
32	Set/Get User Code	🚫	🚫	🚫	✅	✅	✅	✅
33	Read Random	🚫	🚫	🚫	✅	✅	✅	✅
34	Write Random	🚫	🚫	🚫	✅	✅	✅	✅
35	Compute File Size	🚫	🚫	🚫	✅	✅	✅	✅
36	Set Random Record	🚫	🚫	🚫	✅	✅	✅	✅
37	Reset Drives	🚫	🚫	🚫	🚫	✅	✅	✅
38	Access Drive	🚫	🚫	🚫	🚫	✅	✅	✅
39	Free Drive	🚫	🚫	🚫	🚫	✅	✅	✅
40	Write Random with Zero Fill	🚫	🚫	🚫	🚫	✅	✅	✅
41	Test and Write Record	🚫	🚫	🚫	🚫	🚫	✅	✅
42	Lock Record	🚫	🚫	🚫	🚫	🚫	✅	✅
43	Unlock Record	🚫	🚫	🚫	🚫	🚫	✅	✅
44	Set Multi-Sector Count	🚫	🚫	🚫	🚫	🚫	✅	✅
45	Set BDOS Error Mode	🚫	🚫	🚫	🚫	🚫	✅	✅
46	Get Disk Free Space	🚫	🚫	🚫	🚫	🚫	✅	✅

		CP/M Version						
#	Function Name	1975	1.3	1.4	2.0	2.2	3.0 non-banked	3.0 banked
47	Chain to Program	⛔	⛔	⛔	⛔	⛔	✅	✅
48	Flush Buffers	⛔	⛔	⛔	⛔	⛔	✅	✅
49	Set/Get System Control Block	⛔	⛔	⛔	⛔	⛔	✅	✅
50	Direct BIOS Calls	⛔	⛔	⛔	⛔	⛔	✅	✅
59	Load Overlay	⛔	⛔	⛔	⛔	⛔	✅	✅
60	Call Resident System Extension	⛔	⛔	⛔	⛔	⛔	✅	✅
98	Free Blocks	⛔	⛔	⛔	⛔	⛔	✅	✅
99	Truncate File	⛔	⛔	⛔	⛔	⛔	✅	✅
100	Set Directory Label	⛔	⛔	⛔	⛔	⛔	✅	✅
101	Return Directory Label Data	⛔	⛔	⛔	⛔	⛔	✅	✅
102	Read File Date Stamps and Password Mode	⛔	⛔	⛔	⛔	⛔	✅	✅
103	Write File XFCB	⛔	⛔	⛔	⛔	⛔	✅	✅
104	Set Date and Time	⛔	⛔	⛔	⛔	⛔	✅	✅
105	Get Date and Time	⛔	⛔	⛔	⛔	⛔	✅	✅
106	Set Default Password	⛔	⛔	⛔	⛔	⛔	⛔	✅
107	Return Serial Number	⛔	⛔	⛔	⛔	⛔	✅	✅
108	Set/Get Program Return Code	⛔	⛔	⛔	⛔	⛔	✅	✅
109	Set/Get Console Mode	⛔	⛔	⛔	⛔	⛔	✅	✅
110	Set/Get Output Delimiter	⛔	⛔	⛔	⛔	⛔	✅	✅
111	Print Block	⛔	⛔	⛔	⛔	⛔	✅	✅
112	List Block	⛔	⛔	⛔	⛔	⛔	✅	✅
152	Parse Filename	⛔	⛔	⛔	⛔	⛔	✅	✅

4. System Control Block

The System Control Block (SCB) was introduced in CP/M 3.0. It contains the shared BDOS state that CP/M maintains. While the SCB was only used in CP/M 3.0, it is used internally in the Zcim Project virtual CP/M layer for all flavors.

Offset	Id	Description
00H	HashLength	Reserved for system use. hash length (0,2,3)
01H	Hash1	Reserved for system use. hash entry first word
03H	Hash2	Reserved for system use. hash entry second word
05H	BDOSVersion	BDOS Version number
06H	Utility1	Reserved for user use. Use this word for your own flags or data.
08H	Utility2	Reserved for user use. Use this word for your own flags or data.
0AH	DisplayFlag1	Reserved for system use. display flags 1
0CH	DisplayFlag2	Reserved for system use. display flags 2
0EH	CLPFlags	CLP flags
0F	SubmitFileDrive	Submit file drive number
10H	ProgramReturnCode	Program Error Return Code. This 2-byte field can be used by a program to pass an error code or value to a chained program. CP/M 3′s conditional command facility also uses this field to determine if a program executes successfully. The BDOS Function 108 (Get/Set Program Return Code) is used to get/set this value.
12H	MultipleCommandBufferPage	multiple command buffer page
13H	CCPDrive	ccp default drive
14H	CCPUser	ccp default user number
15H	CCPBufferAddress	ccp console buffer address
17H	CCPFlag1	ccp flags byte 1
18H	CCPFlag2	ccp flags byte 2
19H	CCPFlag3	ccp flags byte 3
1AH	ConsoleWidth	Console Width. This byte contains the number of columns, characters per line, on your console relative to zero. Most systems default this value to 79. You can set this default value by using the GENCPM or the DEVICE utility. The console width value is used by the banked version of CP/M 3 in BDOS function 10, CP/M 3′s console editing input function. Note that typing a character into the last position of the screen, as

Offset	Id	Description
		specified by the Console Width field, must not cause the terminal to advance to the next line.
1BH	ConsoleColumn	Console Column Position. This byte contains the current console column position.
1CH	ConsolePageLength	Console Page Length. This byte contains the page length, lines per page, of your console. Most systems default this value to 24 lines per page. This default value may be changed by using the GENCPM or the DEVICE utility.
1DH	ConsoleLine	current console line number
1EH	ConsoleInputBufferAddress	console input buffer address
20H	ConsoleInputBufferLength	console input buffer length
22H	ConsoleInRedirection	console input (CONIN) redirection flag
24H	ConsoleOutRedirection	console output (CONOUT) redirection flag
26H	AuxInRedirection	auxiliary input (AUXIN) redirection flag
28H	AuxOutRedirection	auxiliary output (AUXOUT) redirection flag
2AH	ListOutRedirection	list output (LSTOUT) redirection flag
2CH	PageMode	Page Mode. If this byte is set to zero, some CP/M 3 utilities and CCP built-in commands display one page of data at a time; you display the next page by pressing any key. If this byte is not set to zero, the system displays data on the screen without stopping. To stop and start the display, you can press CTRL-S and CTRL-Q, respectively.
2DH	PageModeDefault	page mode default
2EH	BackspaceFlag	Determines if CTRL-H is interpreted as a rubout/DEL character. If this byte is set to 0, then CTRL-H is a backspace character (moves back and deletes). If this byte is set to 0FFH, then CTRL-H is a rubout/DEL character, echoes the deleted character.
2FH	DeleteFlag	Determines if rubout/DEL is interpreted as CTRL-H character. If this byte is set to 0, then rubout/DEL echoes the deleted character. If this byte is set to 0FFH, then rubout/DEL is interpreted as a CTRL-H character (moves back and deletes).

Offset	Id	Description
30H	TypeAheadFlag	type ahead active
31H	-	console translation subroutine
33H	ConsoleMode	Console Mode. This is a 16-bit system parameter that determines the action of certain BDOS Console I/O functions. console mode (raw/cooked)
35H	BDOSBuffer	128 byte buffer available to banked BIOS
37H	OutputDelimiter	Output delimiter character. The default output delimiter character is $, but you can change this value by using the BDOS Function 110, Get/Set Output Delimiter.
38H	ListOutputFlag	List Output Flag. If this byte is set to 0, console output is not echoed to the list device. If this byte is set to 1 console output is echoed to the list device.
39H	ScrollFlag	queue flag for type ahead
3AH	SCBAddress	system control block address
3CH	DMAAddress	Current DMA Address. This address can be set by BDOS Function 26 (Set DMA Address). The CCP initializes this value to 0080H. BDOS Function 13, Reset Disk System, also sets the DMA address to 0080H.
3EH	Disk	Current Disk. This byte contains the currently selected default disk number. This value ranges from 0-15 corresponding to drives A-P, respectively. BDOS Function 25, Return Current Disk, can be used to determine the current disk value.
3FH	BDOSInfo	BDOS variable “info”. Usually the DE register pair passed as a parameter, but it gets manipulated at times.
41H	resel	disk reselect flag
42H	SameDiskFlag	relog flag
43H	BDOSFunction	function number
44H	UserNumber	Current User Number. This byte contains the current user number. This value ranges from 0-15. BDOS Function 32, Set/Get User Code, can change or interrogate the currently active user number.
45H	NextDirectory	directory record number

Offset	Id	Description
47H	SearchFCB	fcb address for searchn function
49H	SearchType	scan length for search functions
4AH	MultiSectorCount	BDOS Multi-Sector Count. This field is set by BDOS Function 44,, Set Multi-Sector Count.
4BH	BDOSErrorMode	BDOS Error Mode. This field is set by BDOS Function 45, Set BDOS Error Mode. If this byte is set to 0FFH, the system returns to the current program without displaying any error messages. If it is set to 0FEH, the system displays error messages before returning to the current program. Otherwise, the system terminates the program and displays error messages.
4CH	DriveSearch0	Drive Search Chain. The first byte contains the drive number of the first drive in the chain, the second byte contains the drive number of the second drive in the chain, and so on, for up to four bytes. If less than four drives are to be searched, the next byte is set to 0FFH to signal the end of the search chain. The drive values range from 0-16, where 0 corresponds to the default drive, while 1-16 corresponds to drives A-P, respectively.
4DH	DriveSearch1	search chain - 2nd drive
4EH	DriveSearch2	search chain - 3rd drive
4FH	DriveSearch3	search chain - 4th drive
50H	TemporaryFileDrive	Temporary File Drive. This byte contains the drive number of the temporary file drive. The drive number ranges from 0-16, where 0 corresponds to the default drive, while 1-16 corresponds to drives A-P, respectively.
51H	ErrorDrive	Error drive. This byte contains the drive number of the selected drive when the last physical or extended error occurred.
52H	-	Unknown
54H	OpenDoorFlag	drive door open flag
55H	-	Unknown

Offset	Id	Description
57H	BDOSFlags	BDOS Flags. Bit 7 applies to banked systems only. If bit 7 is set, then the system displays expanded error messages. The second error line displays the function number and FCB information. (See Section 2.3.13). Bit 6 applies only to nonbanked systems. If bit 6 is set, it indicates that GENCPM has specified single allocation vectors for the system. Otherwise, double allocation vectors have been defined for the system. Function 98, Free Blocks, returns temporarily allocated blocks to free space only if bit 6 is reset.
58H	DayNumber	Binary number of days since 1 January 1978.
5AH	HoursBCD	Hour in BCD (2-digit Binary Coded Decimal).
5BH	MinutesBCD	Minutes (BCD)
5CH	SecondsBCD	Seconds (BCD)
5DH	CommonMemoryBase	Common Memory Base Address. This value is zero for nonbanked systems and nonzero for banked systems.
5FH	JMP	JMP opcode
60H	BDOSErrorRoutine	BDOS error routine address
62H	BDOSAddress	top of user TPA (address at 6,7)

Table 10.: System Control Block (SCB) Description

Offset	Id	Size	R/W or R/O	Equate
00H	HashLength	Byte	R/O	hashl
01H	Hash1	Word	R/O	hash
03H	Hash2	Word	R/O	hash
05H	BDOSVersion	Byte	R/O	bdos$version
06H	Utility1	Word	R/W	util$flgs
08H	Utility2	Word	R/W	util$flgs
0AH	DisplayFlag1	Word	R/O	dspl$flgs
0CH	DisplayFlag2	Word	R/O	dspl$flgs
0EH	CLPFlags	Byte	R/O	clp$flgs
0FH	SubmitFileDrive	Byte	R/O	clp$drv
10H	ProgramReturnCode	Word	R/W	progretcode
12H	MultipleCommandBufferPage	Byte	R/O	multirsxpg
13H	CCPDrive	Byte	R/O	ccpdrv
14H	CCPUser	Byte	R/O	ccpusr
15H	CCPBufferAddress	Word	R/O	ccpconbuf
17H	CCPFlag1	Byte	R/O	ccpflag1
18H	CCPFlag2	Byte	R/O	ccpflag2
19H	CCPFlag3	Byte	R/O	ccpflag3
1AH	ConsoleWidth	Byte	R/W	conwidth
1BH	ConsoleColumn	Byte	R/O	concolumn
1CH	ConsolePageLength	Byte	R/W	conpage
1DH	ConsoleLine	Byte	R/O	conline
1EH	ConsoleInputBufferAddress	Word	R/O	conbuffer
20H	ConsoleInputBufferLength	Word	R/O	conbuffl
22H	ConsoleInRedirection	Word	R/W	conin$rflg
24H	ConsoleOutRedirection	Word	R/W	conout$rflg
26H	AuxInRedirection	Word	R/W	auxin$rflg
28H	AuxOutRedirection	Word	R/W	auxout$rflg
2AH	ListOutRedirection	Word	R/W	listout$rflg
2CH	PageMode	Byte	R/W	page$mode
2DH	PageModeDefault	Byte	R/O	page$def
2EH	BackspaceFlag	Byte	R/W	ctlh$act
2FH	DeleteFlag	Byte	R/W	rubout$act
30H	TypeAheadFlag	Byte	R/O	type$ahead
31H	-	Word	R/O	contran
33H	ConsoleMode	Word	R/W	con$mode

Offset	Id	Size	R/W or R/O	Equate
35H	BDOSBuffer	Word	R/O	ten$buffer
37H	OutputDelimiter	Byte	R/W	outdelim
38H	ListOutputFlag	Byte	R/W	listcp
39H	ScrollFlag	Byte	R/O	q$flag
3AH	SCBAddress	Word	R/O	scbad
3CH	DMAAddress	Word	R/O	dmaad
3EH	Disk	Byte	R/O	seldsk
3FH	BDOSInfo	Word	R/O	info
41H	FCBErrorFlag	Byte	R/O	resel
42H	SameDiskFlag	Byte	R/O	relog
43H	BDOSFunction	Byte	R/O	fx
44H	UserNumber	Byte	R/O	usrcode
45H	NextDirectory	Word	R/O	dcnt
47H	SearchFCB	Word	R/O	searcha
49H	SearchType	Byte	R/O	searchl
4AH	MultiSectorCount	Byte	R/W	multcnt
4BH	BDOSErrorMode	Byte	R/W	errormode
4CH	DriveSearch0	Byte	R/W	drv0
4DH	DriveSearch1	Byte	R/W	drv1
4EH	DriveSearch2	Byte	R/W	drv2
4FH	DriveSearch3	Byte	R/W	drv3
50H	TemporaryFileDrive	Byte	R/W	tempdrv
51H	ErrorDrive	Byte	R/O	errdrv
52H	-	Word	R/O	-
54H	OpenDoorFlag	Byte	R/O	media$flag
55H	-	Word	R/O	-
57H	BDOSFlags	Byte	R/O	bdos$flags
58H	DayNumber	Word	R/W	date
5AH	HoursBCD	Byte	R/W	-
5BH	MinutesBCD	Byte	R/W	-
5CH	SecondsBCD	Byte	R/W	-
5DH	CommonMemoryBase	Word	R/O	com$base
5FH	JMP	Byte	R/O	error
60H	BDOSErrorRoutine	Word	R/W	error$jmp
62H	BDOSAddress	Word	R/O	top$tpa

Table 11.: SCB Details

5. BIOS Interface

5.1. Pre-BIOS Versions

5.2. Version 2 BIOS

5.3. Version 3 BIOS

5.4. BIOS Data Areas

6. CP/M File System

The CP/M file system was designed around low-memory usage. It provides a limited set of features, and very little in the way of validation or error checking. Since a floppy disk is removable, the focus of error handling was on preventing data loss when floppy disks were removed at unexpected times.

It is a flat file system, with a single level directory structure. Later versions included the concept of a **user** which divided the system into multiple flat file systems, since only the current user's files could be accessed at any time.

Unlike most file systems, it does not keep a list of available blocks (free list) in permanent storage, on the disk itself. While the BDOS/BIOS combination supports multiple physical disk layouts, the file system meta-data does not describe the layout of the disk.

This lead to many inter-operability issues. In practice, the IBM 3740 8″ Single-sided, Single-density floppy disk was the only one likely to work between systems from two different manufactures. Even if they both used the same version of CP/M. One of the downfalls of the CP/M ecosystem was the number of floppy disk formats that had to be maintained by software distributors.

6.1. Floppy Disk Basics

The CP/M operating system was originally designed for the floppy disks of the mid 1970's. The original disk was 8 inches in diameter and was created for an IBM 3740 Data Entry System. This became the standard that was used by the Shugart Associates drives that Gary Kildall used to create CP/M.

It is a single-sided device. It is recorded in what became called "Single-Density" format. It supported 128 byte sectors. With 26 sectors per track, and 77 tracks per disk. This is $128 * 26 * 77 = 256256$ bytes capacity, which is 250KiB.

The original CP/M only supported the "Standard" disk format. The BDOS/BIOS split of the 2.x version added support for more varied disk layouts. Version 2.2 added support for improved sector blocking and deblocking for physical sector sizes that were more than the 128 byte logical sector size.

6.1.1. Disk Layout

Given its history, the BDOS/BIOS interface assumes the original floppy disk like disk layout. It is a number of sectors and tracks of 128 byte sectors.

There is no concept of a side, or a head number. It also assumes that the disk layout is fixed. That is, the number of sectors for the first track is the same as the number of sectors in the last track. While later versions provide some support for efficient blocking and deblocking of non-128 byte sectors, the underlying file system always uses 128 byte logical sectors.

6.1.2. Hard Disk Considerations

The primary hard disk consideration is capacity, or lack thereof. CP/M 2.2 supports disks with up-to 4MiB. CP/M 3.0 supports disks up to 512MiB.

All CP/M drives use the same sectors and tracks model.

It is possible to tell CP/M that the drive cannot be removed so that the directory checksum map for the drive does not need to be allocated.

6.1.3. Sector Blocking and Deblocking

6.1.4. System Limitations

6.2. File Control Blocks

6.3. Directory Entries

6.3.1. Standard Directory Entry

6.3.2. Disk Label Entry

6.3.3. File Date and Time Stamp Entries

6.3.4. Directory Checksums

6.4. Allocation Map

6.4.1. Purpose

6.4.2. Single-Bit Allocation Map

6.4.3. Two-Bit Allocation Map